

Bases de datos

UD2.2. Diseño lógico.
Modelo Relacional



CONTENIDO

1. Modelo Relacional
2. Transformación del modelo E/R al Modelo Relacional.
3. Normalización.

1. Modelo relacional.

1.1. Introducción.

- ▶ Es el modelo más utilizado en la actualidad para modelar problemas reales y administrar datos dinámicamente.
- ▶ Nos aísla de la capa física, es decir, nos evita tener que conocer las estructuras de datos físicas sobre los que se construye la base de datos.
- ▶ El modelo obtenido puede materializarse sobre cualquier gestor de bases de datos relacional.
- ▶ Es independiente de la forma en que se almacenan los datos y de la forma de representarlos (el modelo se puede implementar en cualquier SGBD y los datos se pueden gestionar utilizando cualquier aplicación gráfica).
- ▶ Una vez hemos hecho el diagrama E/R el paso al modelo relacional es prácticamente automático a través de una serie de reglas de transformación.

1. Modelo relacional.

1.1. Introducción.

- ▶ Este modelo considera la base de datos como una colección de **relaciones**. Una relación representa una **tabla**, es decir, una estructura formada por filas y columnas que almacena los datos referentes a una determinada entidad o relación del mundo real.
- ▶ Cada **fila** (tupla o registro) es un conjunto de campos y cada **campo** (atributo) contiene un valor. Cada atributo tiene un **dominio**, que representa todos los posibles valores que puede tomar dicho atributo.
- ▶ Al número de tuplas que tiene una relación se le conoce como **cardinalidad** de la relación y el número de columnas es el **grado** de la relación.
 - ▶ Ej: El grado de la relación es de 5 y la cardinalidad 6.

P#	NOMP	COLOR	PESO	CIUDAD
P1	Tuerca	Rojo	12	Londres
P2	Perno	Verde	17	Paris
P3	Tornillo	Azul	17	Roma
P4	Tornillo	Rojo	14	Londres
P5	Leva	Azul	12	Paris
P6	Rueda	Rojo	19	Londres

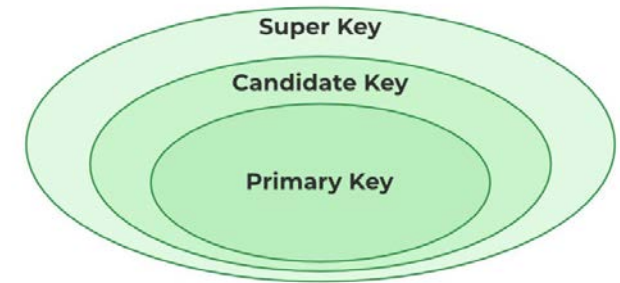
1. Modelo relacional.

1.2. Valor NULO.

- ▶ Es un concepto que se utiliza para definir información desconocida, inaplicable, inexistente, no valida ...
- ▶ Se necesitan cuando:
 - ▶ Tenemos tuplas en las cuales se desconocen algunos valores de sus atributos.
 - ▶ Se necesita añadir algún atributo a una tabla ya existente e, inicialmente, se desconocen los valores de ese atributo para las tuplas.
 - ▶ Puede haber atributos en alguna relación que sólo tengan sentido o sean aplicables en determinadas tuplas.

1. Modelo relacional.

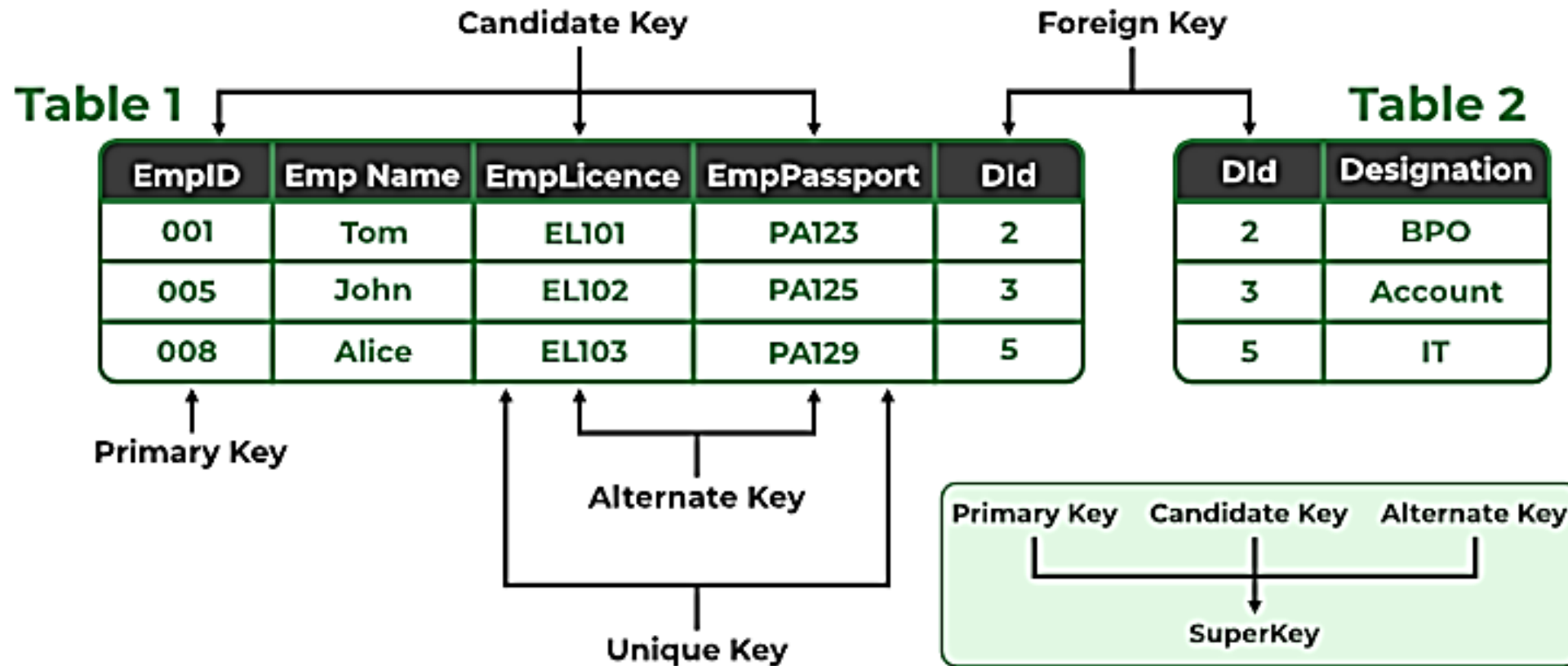
1.3. Claves.



- ▶ **Clave candidata:** es el atributo o conjunto de atributos que identifican unívocamente cada tupla de la relación. Es decir, columnas cuyos valores no se repiten para esa tabla.
- ▶ **Clave primaria (primary key):** es aquella clave candidata que se escoge como identificador de las tuplas. Se escogerá como primaria la clave candidata que identifique mejor a cada tupla en el contexto de la base de datos, pudiendo existir solamente una por tabla. La elección y definición de esta clave conlleva la creación automática de un índice primario por parte del gestor. Cualquier clave candidata que no se haya seleccionado como clave primaria se denomina **clave alternativa**. Estas claves alternativas se denominan también **claves únicas** y la diferencia con la clave primaria es que éstas admiten valores nulos, mientras que una clave primaria nunca puede ser nula.
- ▶ **Clave externa, ajena o foránea (foreign key):** Atributo/s cuyos valores coinciden con una clave candidata (generalmente la clave primaria) de otra tabla y que vincula ambas tablas.
- ▶ **Clave compuesta o superclave:** cuando no hay un valor claro para definir una clave con los atributos que tenemos, podemos optar por crear un nuevo atributo (también llamado **clave artificial**) y hacerlo clave primaria o combinar varios atributos cuya combinación sea muy difícil de repetir, y convertir esta combinación en clave primaria. Por ejemplo, si dudamos del DNI como clave primaria, podríamos usar DNI + Nombre como clave. La probabilidad de que se repita la combinación de estos dos valores es menor de la probabilidad de que se repita sólo el DNI.

1. Modelo relacional.

1.3. Claves.



1. Modelo relacional.

1.4. Integridad.

Restricciones de Integridad

Una restricción es una condición de obligado cumplimiento por los datos de la base de datos. Pueden estar:

- ▶ Definidas por el hecho de que la base de datos sea relacional:
 - ▶ No puede haber dos tuplas iguales
 - ▶ El orden de las tuplas no es significativo
 - ▶ El orden de los atributos no es significativo
 - ▶ Cada atributo sólo puede tomar un valor en el dominio en el que está inscrito (no se admiten atributos multievaluados)
- ▶ Incorporadas por los usuarios:
 - ▶ Clave primaria (primary key)
 - ▶ Unicidad (unique)
 - ▶ Obligatoriedad (not null)
 - ▶ Integridad referencial o clave ajena o foránea (foreign key)

1. Modelo relacional.

1.4. Integridad.

Tipos de restricciones

1. Restricción de unicidad de registro (UNIQUE). No puede haber dos registros con todos los valores de sus atributos iguales en una misma tabla. Cada registro es diferente de otro al menos en uno de sus atributos.

2. Restricción de valor no nulo (NOT NULL). Ninguno de los atributos de una clave primaria en una tabla puede ser nulo o vacío.

3. Restricción de clave primaria (PRIMARY KEY). No puede haber dos registros con la misma clave primaria en una tabla y ésta tampoco puede ser nula (ninguno de los atributos que la componen). (OJO: algunas configuraciones de SGBD distinguen mayúsculas y minúsculas). Es decir una clave primaria es UNIQUE + NOT NULL.

4. Restricción de verificación (CHECK). Comprobaciones sobre una columna para comprobar si su valor es válido conforme a una expresión. *Por ejemplo, podemos prevenir que se graben datos en una columna de Edad mayores de 300 años.*

1. Modelo relacional.

1.4. Integridad.

Tipos de restricciones

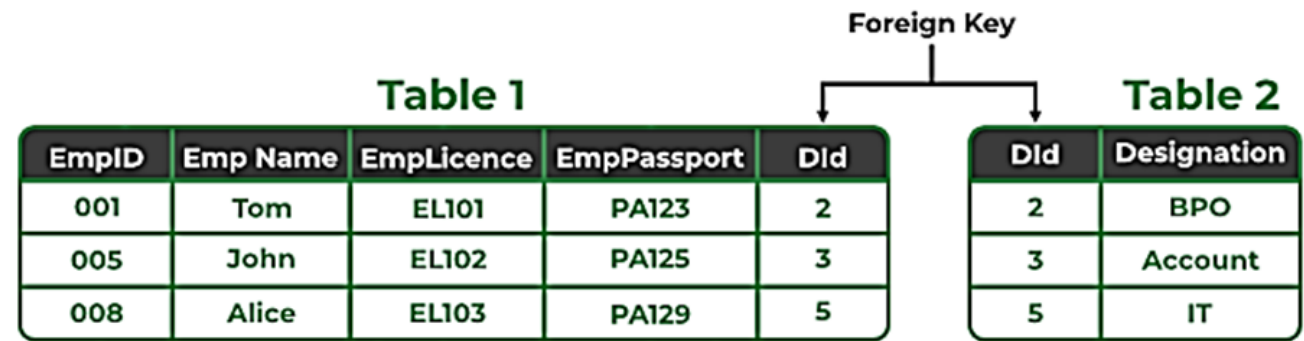
5. Restricción de integridad referencial (FOREIGN KEY). Si sobre un dominio o conjunto de dominios se ha definido en una relación R1, una clave primaria, si estos mismos campos, se encuentran en una relación R2, el valor de estos, será un valor que se encuentra en R1 o su valor será nulo. Por tanto, en R2 estaremos estableciendo una clave externa, que crea por defecto un índice secundario.

Es decir, un valor de un campo en una tabla que es clave externa, siempre debe estar presente entre los valores de ese campo en la tabla en la que esta clave es primaria.

Esto se define así para evitar problemas en las operaciones de borrado y modificación de registros, ya que si se ejecutan esas operaciones sobre la tabla principal quedarán filas en la tabla secundaria con la clave externa sin integridad.

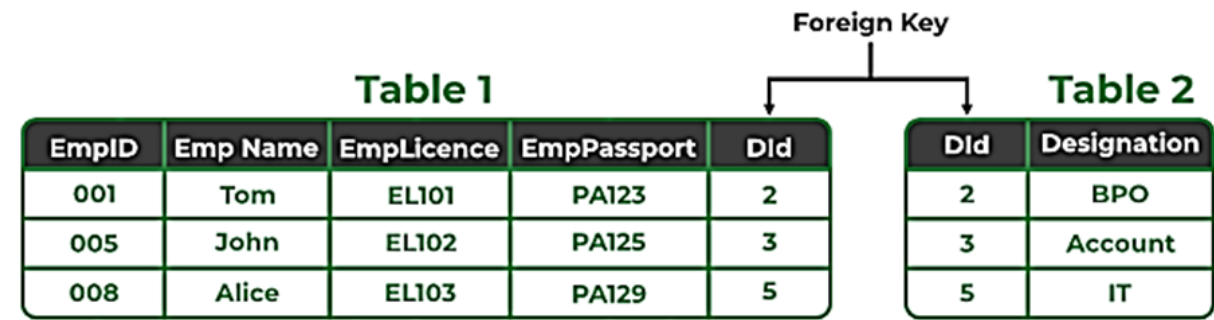
Por ejemplo, si en el ejemplo de arriba borrásemos de la Tabla 2 el registro con DId = 2, tendremos que decidir y decirle a nuestro modelo qué debe hacer con el/los registros de la Tabla 1 donde aparezca este valor (es este caso sólo aquel con EmpID = 001).

A continuación, veremos las acciones que podemos definir ante estas operaciones.



1. Modelo relacional.

1.4. Integridad.



- ▶ CASCADE: borrar o modificar una clave en una fila en la tabla referenciada con un valor determinado de clave, implica borrar las filas con el mismo valor de clave foránea o modificar los valores de esas claves foráneas.

En nuestro ejemplo, al borrar/modificar el registro con DId = 2 de la Tabla 2, se borrarán/modificarán el/los registro/s asociado/s a este registro en la Tabla 1, es decir se borrará/modificará el registro con EmpID = 001.

- ▶ SET NULL: borrar o modificar una clave en una fila en la tabla referenciada con un valor determinado de clave, implica asignar el valor NULL a las claves foráneas con el mismo valor.

En nuestro ejemplo, al borrar/modificar el registro con DId = 2 de la Tabla 2, pondrá a NULL el campo DId de la Tabla 1 asociado con el registro alterado, es decir, se pondrá a nulo el campo DId del registro con EmpID = 001.

- ▶ NO ACTION o RESTRICT: El borrado de las filas de la relación que contiene la clave primaria referenciada, o la modificación de dicha clave, sólo se permite si no existen filas con dicha clave en la relación que contiene la clave ajena. Es decir, no puedo borrar en la tabla de la clave primaria si en una tabla con clave foránea referencio a ese valor.

En nuestro ejemplo no nos permitirá borrar/modificar el registro con DId = 2 de la Tabla 2 ya que existe un registro en la Tabla 1 en el que se hace referencia al mismo, el registro con EmpID = 001.

- ▶ SET DEFAULT: borrar o modificar una clave en una fila en la tabla referenciada con un valor determinado implica asignar el valor por defecto (previamente definido al crear la tabla en el modelo físico) a las claves foráneas con el mismo valor.

En nuestro ejemplo, imaginemos que este valor predefinido es 0. Al borrar/modificar el registro con DId = 2 de la Tabla 2, asignará el VALOR POR DEFECTO previamente definido al campo DId de la Tabla 1 asociado con el registro alterado, es decir, pondrá un 0 en el campo DId del registro con EmpID = 001.

2. TRANSFORMACIÓN DEL MODELO E/R AL MODELO RELACIONAL

2.1. Entidades.

2.2. Atributos.

2.3. Relaciones.

2.4. Ejemplos.

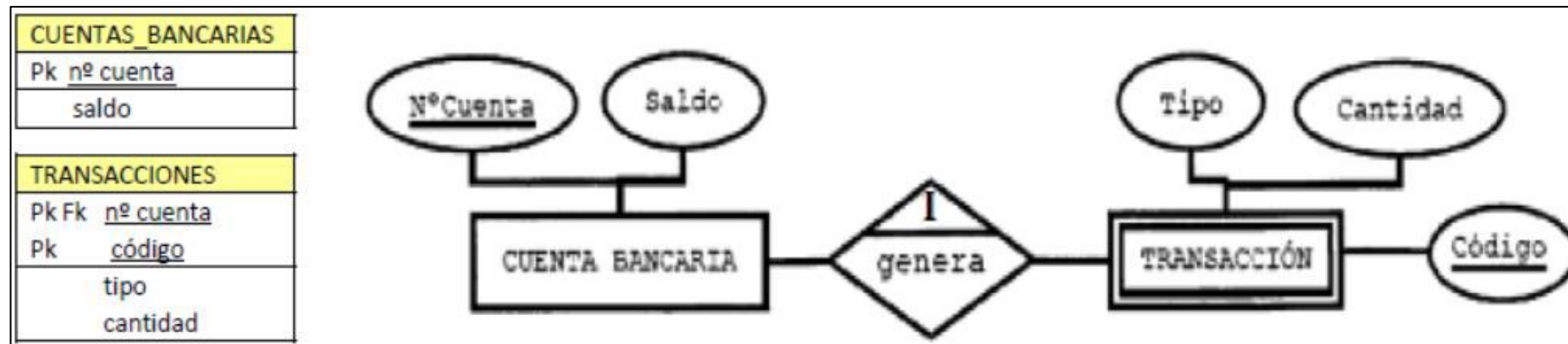
2.5. Transformación en el modelo extendido.

2.1. Entidades.

Las reglas son las siguientes:

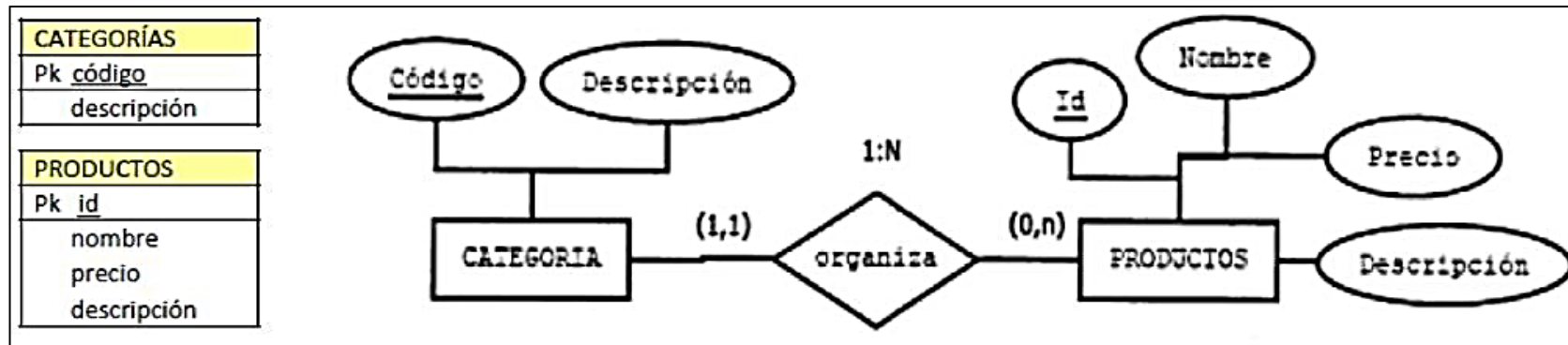
- **Entidades.** Cada entidad pasa a ser una tabla. El identificador único de esa entidad pasa a ser clave primaria y aparece subrayada.

Las entidades débiles también se transforman en tablas, pero cuando tienen dependencia de identificación su clave primaria se compone de la unión de ésta con la clave de la entidad fuerte de la que depende.



2.2. Atributos.

- **Atributos.** Todos los atributos de cada entidad se convertirán en columnas de la tabla. Si hay atributos que se pueden calcular a partir de otros, no los almacenaremos. Por ejemplo, si en persona tenemos la edad y la fecha de nacimiento, solo almacenaremos la fecha de nacimiento. Procuramos guardar el atributo que no cambie con el tiempo.

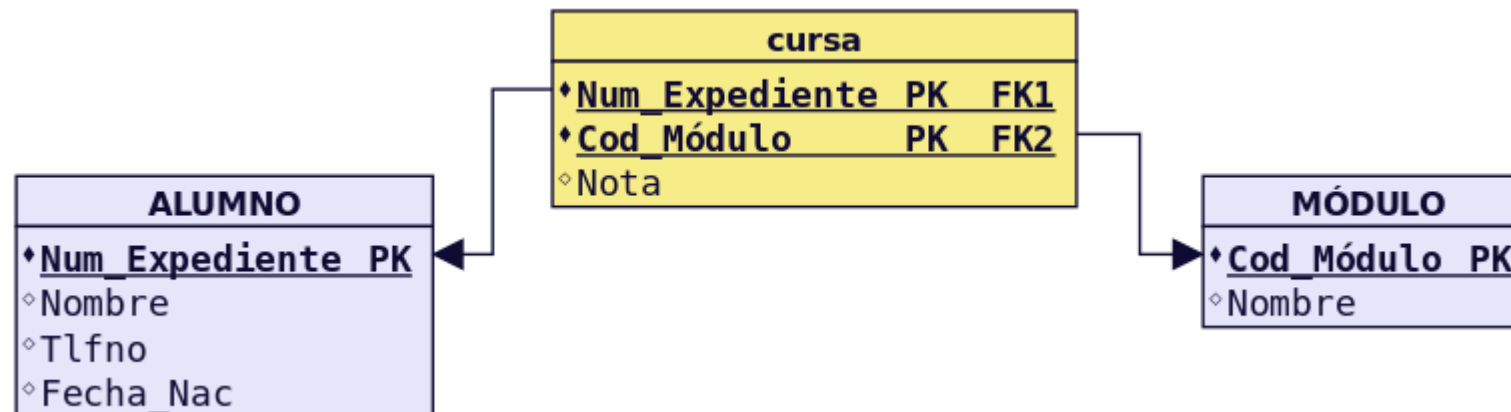


ATRIBUTO IDENTIFICADOR ENTIDAD	PRIMARY KEY
ATRIBUTO IDENTIFICADOR ALTERNATIVO	UNIQUE, NOT NULL
RESTO DE ATRIBUTOS	Restricciones opcionales (dominio - Check, ...)

2.3. Relaciones.

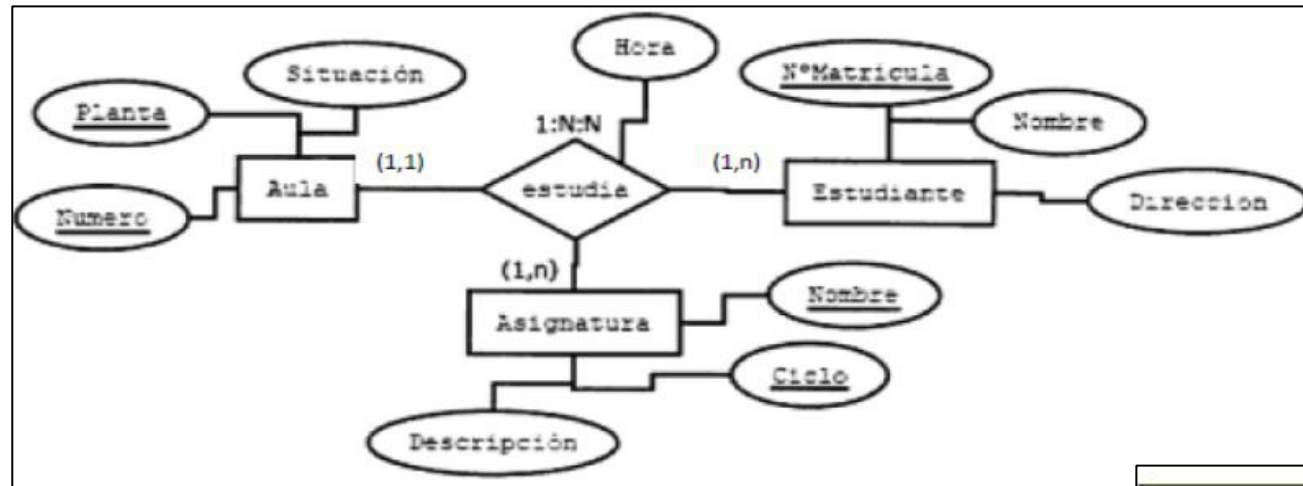
Las reglas son las siguientes:

- **Relaciones N:M.** Siempre generan tabla. Se crea una tabla que incorpora como claves ajenas o foráneas cada una de las claves de las entidades que participan en la relación. La clave principal de esta nueva tabla está compuesta por el conjunto de dichos campos. El orden de los atributos que forman la clave lo determinan las consultas que se vayan a realizar. Las tuplas de la tabla suelen estar ordenadas siguiendo como índice de búsqueda la clave. Por eso, conviene poner primero aquel/los atributos por los que se va a realizar las consultas.



2.3. Relaciones.

- **Relaciones M:N:P.** Al igual que con las N:M, se crea una tabla que incorpora como claves ajenas o foráneas cada una de las claves de las entidades que participan en la relación. La clave principal de esta nueva tabla está compuesta por el conjunto de dichos campos. El orden de los atributos que forman la clave lo determinan las consultas que se vayan a realizar. Las tuplas de la tabla suelen estar ordenadas siguiendo como índice de búsqueda la clave. Por eso, conviene poner primero aquel/los atributos por los que se va a realizar las consultas.



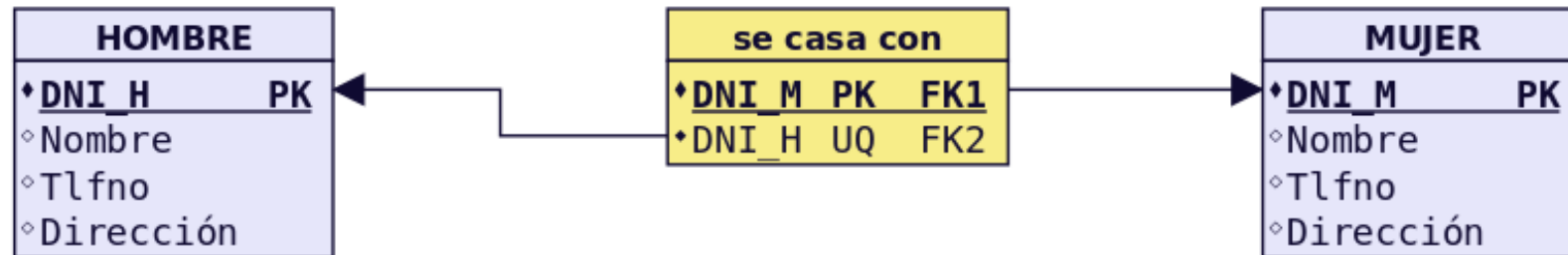
ESTUDIOS	
Pk Fk	<u>número</u>
Pk Fk	<u>planta</u>
Pk Fk	<u>nº matrícula</u>
Pk Fk	<u>nombre</u>
Pk Fk	<u>ciclo</u>
	hora

AULAS	ESTUDIANTES	ASIGNATURAS
Pk <u>número</u>	Pk <u>nº matrícula</u>	Pk <u>nombre</u>
Pk <u>planta</u>	nombre	Pk <u>ciclo</u>
situación	dirección	descripción

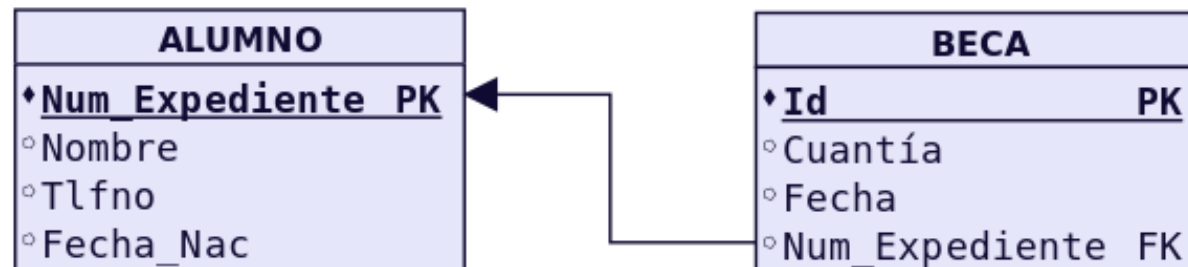
2.3. Relaciones.

► Relaciones 1:1. Generalmente no generan tabla. Hay 2 casos:

- Caso 1: Si las dos entidades participan con participación (0,1), entonces se crea una nueva tabla para la relación.



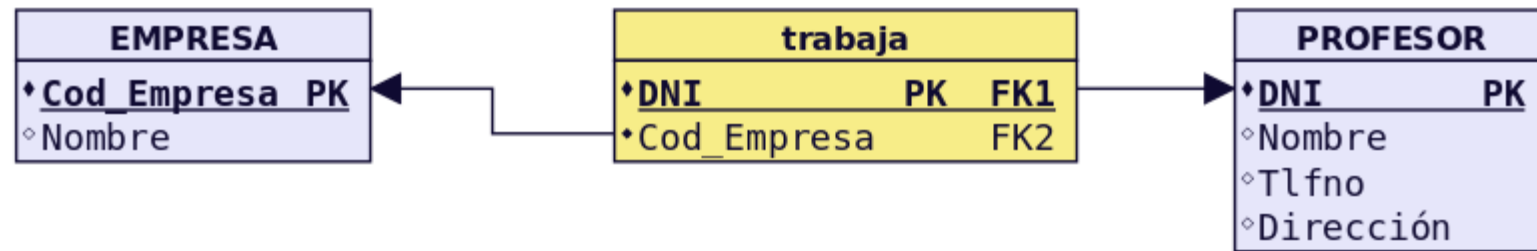
- Caso 2: Si alguna entidad (no las dos), participa con participación mínima 0 (0,1), o las dos tienen (1,1) entonces se pone la clave ajena en dicha entidad, para evitar en lo posible, los valores nulos. En este último caso se podría escoger dónde colocar la clave ajena.



2.3. Relaciones.

► **Relaciones 1:N.** Generalmente no generan tabla. Hay 2 casos:

- Caso 1: Si la entidad del lado "1" presenta participación (0,1), entonces se crea una nueva tabla para la relación que incorpora como claves ajenas las claves de ambas entidades. La clave principal de la relación será sólo la clave de la entidad del lado "N".



- Caso 2: Para el resto de situaciones, la entidad del lado "N" recibe como clave ajena la clave de la entidad del lado "1". Los atributos propios de la relación pasan a la tabla donde se ha incorporado la clave ajena.



2.3. Relaciones.

► Relaciones reflexivas o recursivas.

- Generan tabla o no en función de la cardinalidad. Si es 1:1, no genera tabla. En la entidad se introduce dos veces la clave, una como clave principal y otra como clave ajena. Se suele introducir una modificación en el nombre por diferenciarlas. Si es 1:N, se puede generar tabla o no. Si hubiese participación 0 en uno de los extremos, lo más eficiente es generar una tabla. Si es N:N, la relación genera tabla.

► *Una persona está casada con otra persona, y sólo con una.
Sólo hay personas casadas.*



► *Tenemos un grupo de alumnos en un curso. De cada alumno se desea guardar el nº de expediente, nombre, teléfono, fecha de nacimiento. Cada grupo de alumnos tiene un grupo de alumnos, uno de los cuales es el delegado del grupo, por lo que todos los niños tienen delegado, incluso el propio delegado.*



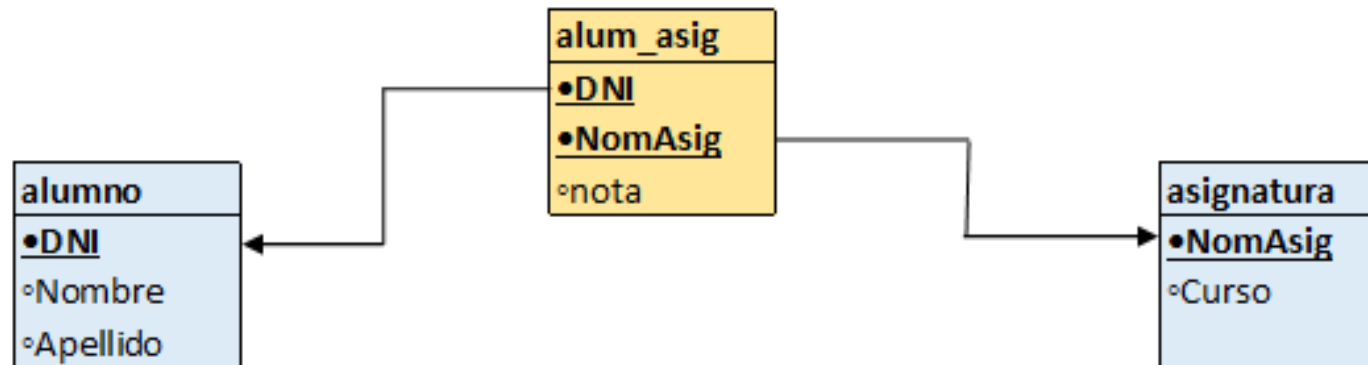
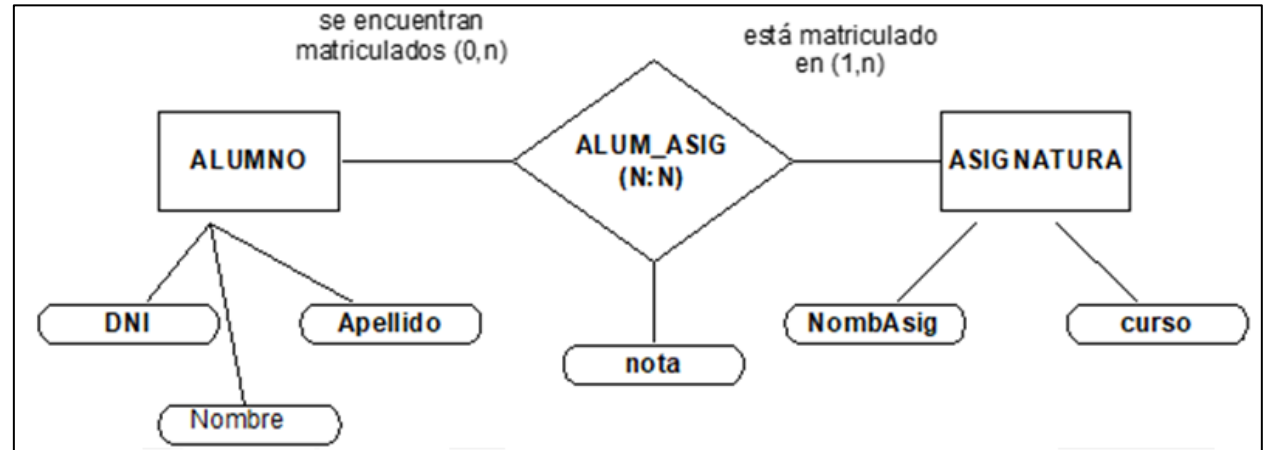
2.4. Ejemplos (I).

En nuestro ejemplo del alumno y las asignaturas, quedaría:

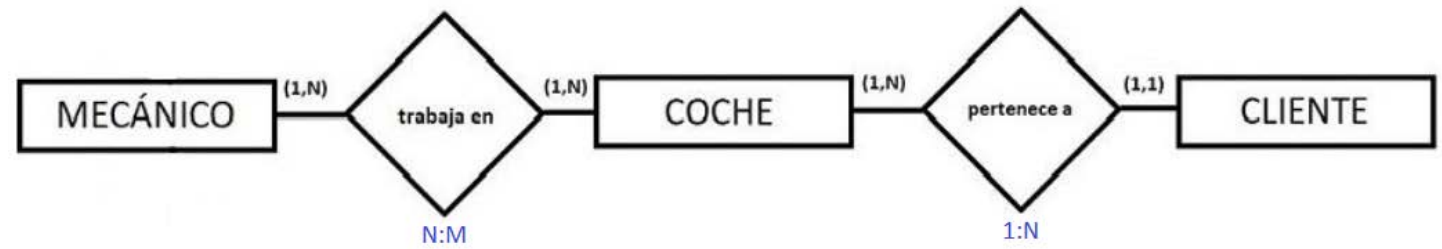
ALUMNO (DNI, nombre, apellido)

ASIGNATURA (NomAsig, curso)

ALUM_ASIG (DNI, NomAsig, Nota)



2.4. Ejemplos(II).



► Volvemos al ejemplo del taller:

- COCHE (matrícula, marca, modelo, color)
 - CLIENTE (dni, nombre, dirección, teléfono, email)
 - MECANICO (Código Empleado, Nombre, Especialidad)
- La relación trabaja en, al ser N:M, genera una tabla que vamos a llamar TRABAJOS, que recogerá los trabajos que realizan los mecánicos en los distintos coches. En cuanto a la clave, está formada por la combinación de dos campos, el código del mecánico y la matrícula de el coche.

Cod_Empleado	Nombre	Especialidad
1	Luis	Motor
2	Paco	Revisiones
3	Esther	Chapa

clave foránea

TRABAJOS

Cod_empleado	Matrícula	Reparación	Piezas	Importe
1	2222WWW	Arreglos varios	7	300,00
2	1111QQQ	Revisión	2	125,00
2	3333PPP	Revisión	4	150,00
3	1111QQQ	Arreglo araño	1	50,00
3	2222WWW	Arreglo capó	1	200,00

clave foránea

DNI	Nombre	Dirección	Telefono	email
111111111R	Pepe	Lope de vega, 3	837694	e@e.com
222222222M	María	Atocha, 6	3374638	
333333333K	Daniel	Mallorca, 7	78732	

clave foránea

COCHE

Matrícula	Marca	Modelo	Color	DNI cliente
1111QQQ	Toyota	Auris	Azul	111111111R
2222WWW	Ford	KA	Verde	222222222M
3333PPP	Seat	Ibiza	Amarillo	333333333K

- TRABAJOS (Código Empleado, matrícula, reparación, piezas, importe)

2.4. Ejemplos(II).

- ▶ ¿Qué ocurre en este modelo? ¿ Y si un mecánico trabajar varias veces en distintos días en el mismo coche? ¿Cómo podemos arreglarlo?
- ▶ TRABAJOS (Código Empleado, matrícula, fecha, reparación, piezas, importe)



2.5. Transformación en el modelo extendido.

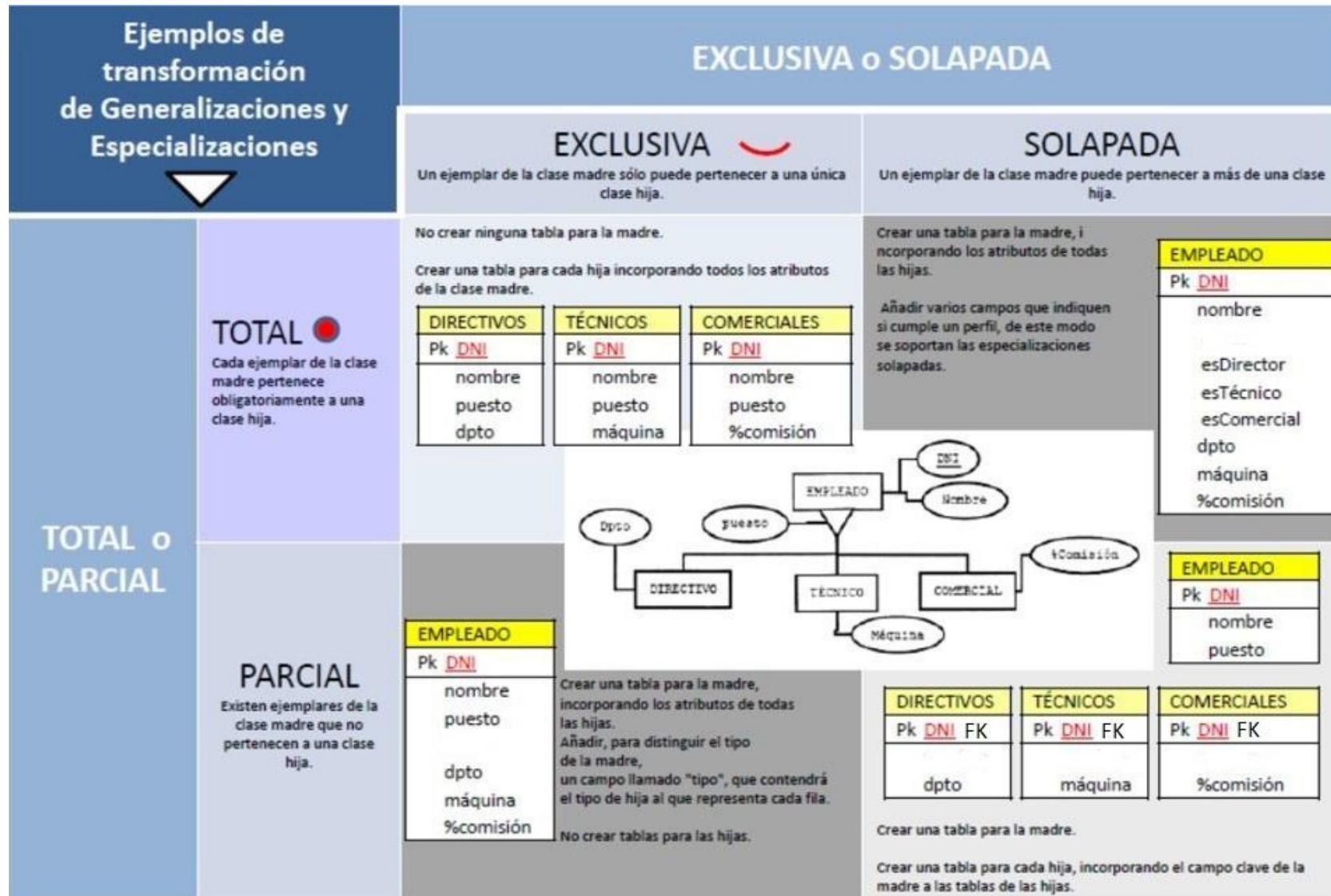
Como los TIPOS y los SUBTIPOS no tienen una representación directa en el Modelo Relacional, se puede optar por diferentes soluciones:

1. Crear una sola relación. → R (supertipo + subtipos)
2. Crear tres relaciones. → R1(supertipo)
R2(subtipo1) R3(subtipo2)
3. Crear dos relaciones. → R1(subtipo1)
R2(subtipo2)

2.5. Transformación en el modelo extendido.

Reglas de transformación para Generalizaciones y Especializaciones 		EXCLUSIVA o SOLAPADA	
		EXCLUSIVA 	SOLAPADA
TOTAL o PARCIAL	TOTAL 	No crear ninguna tabla para la madre. Crear una tabla para cada hija incorporando todos los atributos de la clase madre.	Crear una tabla para la madre, incorporando los atributos de todas las hijas. Añadir varios campos que indiquen si cumple un perfil, de este modo se soportan las especializaciones solapadas.
	PARCIAL	Crear una tabla para la madre, incorporando los atributos de todas las hijas. Añadir, para distinguir el tipo de la madre, un campo llamado "tipo", que contendrá el tipo de hija al que representa cada fila. No crear tablas para las hijas.	Crear una tabla para la madre. Crear una tabla para cada hija, incorporando el campo clave de la madre a las tablas de las hijas.

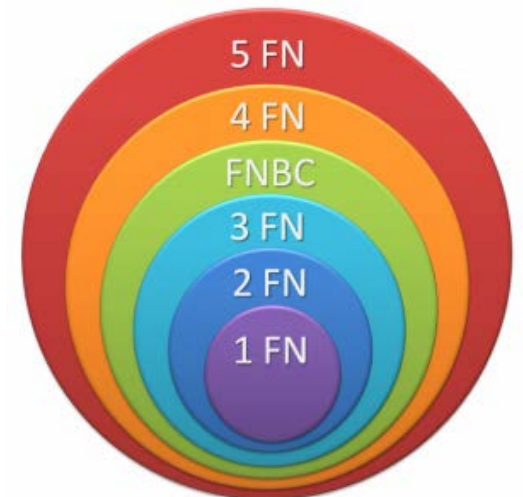
2.5. Transformación en el modelo extendido.



3. NORMALIZACIÓN

3. Normalización.

- ▶ Cuando se diseña una BD, es posible, aunque no muy recomendable, obtener el esquema relacional sin realizar ese paso intermedio que es el esquema conceptual (E/R). Por este motivo es conveniente (obligatorio en el modelo relacional directo) aplicar un conjunto de reglas, conocidas como **Teoría de normalización**, para que el modelo sea menos vulnerable a inconsistencias y anomalías.
- ▶ En la práctica, si la BD se ha diseñado haciendo uso de modelos semánticos como el modelo E/R no suele ser necesaria la normalización. Por otro lado si nos proporcionan una base de datos creada sin realizar un diseño previo, es muy probable que necesitemos normalizar.
- ▶ Las **formas normales (FN)** proporcionan los criterios para determinar el grado de vulnerabilidad de una tabla a inconsistencias y anomalías lógicas. Cuanto más alta sea la forma normal aplicable a una tabla, menos vulnerable será a inconsistencias y anomalías. Cada forma normal incluye a las anteriores.



3. Normalización.

Conceptos previos

- ▶ Dependencia funcional: $A \rightarrow B$, significa que B es funcionalmente dependiente de A. Para un valor de A siempre aparece un valor de B. Ejemplo: Si A es el D.N.I., y B el Nombre, está claro que para un número de D.N.I, siempre aparece el mismo nombre de titular.
- ▶ Dependencia funcional completa: $A \rightarrow B$, si B depende de A en su totalidad. Ejemplo: Tiene sentido plantearse este tipo de dependencia cuando A está compuesto por más de un atributo. Por ejemplo, supongamos que A corresponde al atributo compuesto: D.N.I._Empleado + Cod._Dpto. y B es Nombre_Dpto. En este caso B depende del Cod_Dpto., pero no del D.N.I._Empleado. Por tanto no habría dependencia funcional completa.
- ▶ Dependencia transitiva: $A \rightarrow B \rightarrow C$. Si $A \rightarrow B$ y $B \rightarrow C$, Entonces decimos que C depende de forma transitiva de A. Ejemplo: Sea A el D.N.I. de un alumno, B la localidad en la que vive y C la provincia. Es un caso de dependencia transitiva $A \rightarrow B \rightarrow C$.
- ▶ Determinante funcional: todo atributo, o conjunto de ellos, de los que depende algún otro atributo. Ejemplo: El D.N.I. es un determinante funcional pues atributos como nombre, dirección, localidad, etc, dependen de él.
- ▶ Dependencia multivaluada: $A \twoheadrightarrow B$. Son un tipo de dependencias en las que un determinante funcional no implica un único valor, sino un conjunto de ellos. Un valor de A siempre implica varios valores de B. Ejemplo: CursoBachillerato $\twoheadrightarrow$ Modalidad. Para primer curso siempre va a aparecer en el campo Modalidad uno de los siguientes valores: Ciencias, Humanidades/Ciencias Sociales o Artes. Igual para segundo curso.

3. Normalización.

► Primera Forma Normal: 1FN

Una Relación está en 1FN si y sólo si cada atributo es atómico (cada campo tiene un único valor).

*Ej: un campo **tfno** de una tabla P de personas contiene una lista de valores (teléfonos).*

Alumnos

DNI	Nombre	Curso	FechaMatrícula	Tutor	LocalidadAlumno	ProvinciaAlumno	Teléfonos
11111111A	Eva	1ESO-A	01-Julio-2016	Isabel	Écija	Sevilla	660111222
22222222B	Ana	1ESO-A	09-Julio-2016	Isabel	Écija	Sevilla	660222333 660333444 660444555

Solución → generar una tabla adicional con la clave primaria compuesta por la clase primaria de P + tfno.

Alumnos

DNI	Nombre	Curso	FechaMatrícula	Tutor	LocalidadAlumno	ProvinciaAlumno
11111111A	Eva	1ESO-A	01-Julio-2016	Isabel	Écija	Sevilla
22222222B	Ana	1ESO-A	09-Julio-2016	Isabel	Écija	Sevilla

Teléfonos

DNI	Teléfono
11111111A	660111222
22222222B	660222333
22222222B	660333444
22222222B	660444555

3. Normalización.

► Segunda Forma Normal: 2FN

Una Relación esta en 2FN si y sólo si está en 1FN y todos los atributos que no forman parte de la Clave Principal tienen dependencia funcional completa de ella

Ej: tenemos la tabla

Alumnos

DNI	Nombre	Curso	FechaMatrícula	Tutor	LocalidadAlumno	ProvinciaAlumno
11111111A	Eva	1ESO-A	01-Julio-2016	Isabel	Écija	Sevilla
22222222B	Ana	1ESO-A	09-Julio-2016	Isabel	Écija	Sevilla
33333333C	Susana	1ESO-B	11-Julio-2016	Roberto	Écija	Sevilla

3. Normalización.

Siempre que aparece un DNI aparecerá el Nombre correspondiente y la LocalidadAlumno correspondiente. Por tanto $\text{DNI} \rightarrow \text{Nombre}$ y $\text{DNI} \rightarrow \text{LocalidadAlumno}$.

Por otro lado siempre que aparece un Curso aparecerá el Tutor correspondiente. Por tanto $\text{Curso} \rightarrow \text{Tutor}$.

Los atributos Nombre y LocalidadAlumno no dependen funcionalmente de Curso, y el atributo Tutor no depende funcionalmente de DNI.

El único atributo que sí depende de forma completa de la clave compuesta DNI y Curso es FechaMatrícula: $(\text{DNI}, \text{Curso}) \rightarrow \text{FechaMatrícula}$.

Solución:

Alumnos

DNI	Nombre	Localidad	Provincia
11111111A	Eva	Écija	Sevilla
22222222B	Ana	Écija	Sevilla
33333333C	Susana	El Villar	Córdoba

Matrículas

DNI	Curso	FechaMatrícula
11111111A	1ESO-A	01-Julio-2016
22222222B	1ESO-A	09-Julio-2016
33333333C	1ESO-B	11-Julio-2016

Cursos

Curso	Tutor
1ESO-A	Isabel
1ESO-B	Roberto
2ESO-A	Federico

3. Normalización.

► Tercera Forma Normal: 3FN

Una Relación esta en 3FN si y sólo si está en 2FN y no existen dependencias transitivas. Todas las dependencias funcionales deben ser respecto a la clave principal.

Ej: tenemos la tabla

Alumnos

DNI	Nombre	Localidad	Provincia
11111111A	Eva	Écija	Sevilla
22222222B	Ana	Écija	Sevilla
33333333C	Susana	El Villar	Córdoba

Existe una dependencias funcional transitiva: $DNI \rightarrow Localidad \rightarrow Provincia$.

Solución:

Alumnos

DNI	Nombre	Localidad
11111111A	Eva	Écija
22222222B	Ana	Écija
33333333C	Susana	El Villar

Localidades

Localidad	Provincia
Écija	Sevilla
El Villar	Córdoba

3. Normalización.

► Forma Normal de Boyce-Codd: FNBC

Una Relación esta en FNBC si está en 3FN y no existe solapamiento de claves candidatas.

Ej: tenemos la tabla

Suministros

CIF	Nombre	CódigoPieza	CantidadPiezas
S-11111111A	Ferroman	1	10
B-22222222B	Ferrotex	1	7
S-11111111A	Ferroman	3	15

El atributo CantidadPiezas tiene dependencia funcional de dos claves candidatas: (NombreProveedor, CódigoPieza) y (CIFProveedor, CódigoPieza)

Solución:

Proveedores

CIF	Nombre
S-11111111A	Ferroman
B-22222222B	Ferrotex

Suministros

CIF	CódigoPieza	CantidadPiezas
S-11111111A	1	10
B-22222222B	1	7
S-11111111A	3	15

3. Normalización.

► Cuarta Forma Normal: 4FN

Una Relación esta en 4FN si y sólo si está en 3FN y las únicas dependencias multivaluadas son aquellas que dependen de las claves candidatas.

Ej: tenemos la tabla

Alumnado

Estudiante	Asignatura	Deporte
11111111A	Matemáticas, Lengua	Baloncesto
22222222B	Matemáticas	Fútbol, Natación

Existen dos dependencias multivaluadas: Estudiante $\twoheadrightarrow$ Asignatura y Estudiante $\twoheadrightarrow$ Deporte.

Solución:

EstudiaAsignatura

Estudiante	Asignatura
11111111A	Matemáticas
11111111A	Lengua
22222222B	Matemáticas

PracticaDeporte

Estudiante	Deporte
11111111A	Baloncesto
22222222B	Fútbol
22222222B	Natación

3. Normalización.

► Quinta Forma Normal: 5FN

También conocida como forma normal de proyección-uniión (PJ/NF). Una tabla se dice que está en 5NF si y sólo si está en 4NF y cada dependencia de unión (join) en ella es implicada por las claves candidatas.

Ej: tenemos la tabla

Suministros

Proveedor	Pieza	Proyecto
E1, E4	PI3, PI6	PR2, PR4
E2, E5	PI1, PI2	PR1, PR3

Existen varias dependencias multivaluadas: $\text{Proveedor} \twoheadrightarrow \text{Pieza}$, $\text{Pieza} \twoheadrightarrow \text{Proyecto}$ y $\text{Proyecto} \twoheadrightarrow \text{Proveedor}$.

Solución: creamos una tabla por cada dependencia.

3. Normalización.

Proveedor-Pieza

Proveedor	Pieza
E1	PI3
E1	PI6
E4	PI3
E2	PI1
E2	PI2
E5	PI1
E5	PI2

Pieza-Proyecto

Pieza	Proyecto
PI3	PR2
PI3	PR4
PI6	PR2
PI6	PR4
PI1	PR1
PI1	PR3
PI2	PR1
PI2	PR3

Proyecto-Proveedor

Proyecto	Proveedor
PR2	E1
PR4	E1
PR2	E4
PR4	E4
PR1	E2
PR3	E2
PR1	E5
PR3	E5

Preguntas

